Structures de données

Arfaoui Ahlem

Les structures(1):

Introduction:

- Le **tableau** permet de désigner sous un seul nom un ensemble de valeurs **de même type**, repérées par des indices.
- Si on souhaite stocker des données qui n'ont pas forcément le même type au sein d'une même structure.
- **Exemple:** les informations liées à un produit. Chaque produit est définies par:
  - Nom
  - Prix
  - Numéro…

➔ Les tableaux ne sont pas une solution adéquate.

➔ Le langage C nous permet de créer nos propres types de variables.

➔ Des « **types de variables personnalisés** » ➔ **les structures.**

➔ Ce nouveau type ou cette une nouvelle **structure** permet de **représenter des données hétérogènes** (c'est-à-dire de différents types).

Les structures(2):

Définition:

- Un enregistrement, appelé **structure** En langage C, est composé **d'un nombre fixe d'éléments** défini par l'utilisateur qui peuvent être de **types différents**.
- La liste des éléments est composée de définitions identiques à celle de variables.
- Chaque élément de la structure est nommé **champ.**
- L'accès à un champ se fait **par son nom** dans la structure .
- Les structures sont définies en utilisant le mot-clé **struct.**
- Lors de la déclaration d'une structure, on précise:
  - le nom de chaque champ
  - le type
- Les structures peuvent être déclarées:
  - au niveau du programme principal
  - dans un fichier header à part (sans oublier d'inclure ce fichier dans le programme principal) .

Les structures(3):

Déclaration:

Syntaxe:

Struct <nom_structure>{

<type_champ1><Nom_champ1>;

<type_champ2><Nom_champ2>;

……..

<type_champN><Nom_champN>;

};

Exemple:

```
struct Etudiant {
char Nom[30];
int Age;
float MoyenneScolaire;
};
```

Les structures(4):

Définition d'une variable de type structure :

❖ Une variable structurée doit être définie comme suit :

Struct <nom_structure> <nom_variable>;

Exemple:

```
#include<stdio.h>
struct Etudiant {
char Nom[12];
int Age;
float MoyenneScolaire;
};
int main(){
struct Etudiant E1, E2;
return 0;
}
```

Les structures(5):

Initialisation d'une structure:

❖ En mémoire, en déclarant une variable E1, l'espace réservé aura l'allure suivante:



❖ Lors de sa déclaration on peut initialiser une structure en indiquant entre {}.
❖ La liste des valeurs séparées par des virgules.
❖ Chaque valeur étant une constante ayant le type du champ correspondant.

Les structures(6):

Initialisation d'une structure:

Exemple:

```
#include<stdio.h>
struct Etudiant {
char Nom[12];
int Age;
float MoyenneScolaire;
};
int main(){
struct Etudiant E1={"Ahmed", 21, 13};
struct Etudiant E2={"Mohamed", 24, 15};
return 0;
}
```

➔ **Utilisation de « typedef »**

Typedef: c'est un mot clé réservé dans la langage de programmation C. Il est utilisé pour créer un nom d'alias (un nouveau nom) par un autre type de données.

Les structures(7):

Initialisation d'une structure:

Syntaxe:

Typedef type_de_données alias;

Exemple:

Typedef struct Etudiant E1;

➔
```
#include<stdio.h>
struct Etudiant {
char Nom[12];
int Age;
float MoyenneScolaire;
};
Typedef struct Etudiant Etudiant;
int main(){
Etudiant E1={"Ahmed", 21, 13};
Etudiant E2={"Mohamed", 24, 15};
return 0; }
```

Les structures(8):

Utilisation d'une structure:

- Chaque variable de type structure possède des champs repérés avec des noms uniques.

- Pour accéder aux champs d'une structure on utilise l'opérateur de champ (un simple point) placé entre le nom de la variable structurée et le nom du champ.

Syntaxe:

<nom_variable-struct>.<nom_champi>;

- Ainsi, pour affecter des valeurs à la variable E1, on pourra écrire:

Exemple:

Les structures(9):

Utilisation d'une structure:

Exemple:

```
#include<stdio.h>
struct Etudiant {
char Nom[12];
int Age;
float MoyenneScolaire;
};
int main(){
struct Etudiant E1;
printf("donner la moyeunne pour l'etudiant E1:\n ");
scanf ("%f", &E1.MoyenneScolaire) ;
E1.Age = 50;
printf ("Moyenne = %f\n", E1.MoyenneScolaire) ;
printf ("Age = %d", E1.Age) ;
return 0;
}
```

```
donner la moyeunne pour l'etudiant E1:
 15
Moyenne = 15.000000
Age = 50
Process returned 0 (0x0)   execution time : 5.547 s
Press any key to continue.
```

Les structures(10):

Utilisation d'une structure:

➔ Il est clair, qu'on peut effectuer des affectations entre deux variables de même type de structure:

✓ Soit d'une façon individuelle (champs par champs), sur chacun de leurs champs.

✓ Soit d'une manière globale sur toute la structure.

Exemple:

❖ **Affectation individuelle :**

```
E1. Age = E2.Age;
E1.MoyenneScolaire = E2.MoyenneScolaire;
```

❖ **Affectation Globale :**

```
E1 = E2;  ➔  {  Strcpy(E1.Nom,E2.nom);
                E1. Age = E2.Age;
                E1.MoyenneScolaire = E2.MoyenneScolaire;
```

Les structures(11):

Exercice d'application :

Définir une fiche Livre.

Saisir dans le programme les informations d'un livre et les affichées.

Informations: Titre, Auteur, Edition, Prix.

Les structures(12):

Structures en tant qu'arguments de fonction:

❖ Vous pouvez passer une structure en tant qu'argument à une fonction comme vous passez une variable.

Exemples:

```
#include <stdio.h>
#include <string.h>
struct Livres {
   char  titre[50];
   char  auteur[50];
   char  sujet[100];
   int   livre_id;
};
typedef struct Livres livre;
/void afficherLivre( livre livre1 );
```

```
int main( ) {
livre livre1strcpy( livre1.titre, "Programmation C");
   strcpy( livre1.auteur, "Yaka MainProgramming");
   strcpy( livre1.sujet, "Les structures en C");
   livre1.livre_id= 6495407;
    afficherLivre( livre1 );
}
void afficherLivre(livre livre1 ) {
   printf( "Livre titre: %s\n", livre1.titre);
   printf( "Livre auteur: %s\n", livre1.auteur);
   printf( "Livre sujet: %s\n", livre1.sujet);
   printf( "Livre livre_id: %d\n", livre1.livre_id);
}
```

Imbrication des structures(1):

Structure comportant des tableaux:

❖ Une **structure** peut contenir des champs de type **chaîne de caractères** ou bien de type **tableau**.

Exemple:

```
struct UnEtudiant {
char nom [30];
char prenom [30];
float notes [4];
int niveau;
float moyenne; };
typedef  struct UnEtudiant Etudiant;
```

Notation:

✓ **Etudiant E1, E2;**

✓ **E1.nom[0]** : désigne le premier caractère du champ nom de l'étudiant Etudiant1.

✓ **E2.notes[3]** : désigne la quatrième note du tableau notes de l'étudiant Etudiant2 .

Imbrication des structures(2):

Structure comportant des tableaux:

Initialisation:

```
Etudiant E1 = {"Ben Salem", "Ali", {10.0, 12.5, 13.6, 7.5}, 2, 10.9};
Etudiant E2 = {"Selmi ", "Ahmed", {12.0, 12.5, 14.0, 11.5}, 2, 12.5};
```

Utilisation:

```
strcpy (E1.nom, "Ben Salem");
E1.notes[2] = 13.6;
printf (“%f “, E2.notes[i]);
```

Exemple:

Imbrication des structures(3):

Structure comportant des tableaux:

```
#include<stdio.h>
struct UnEtudiant {
char nom [30];
char prenom [30];
float notes [4];
int niveau;
float moyenne; };
typedef  struct UnEtudiant Etudiant;
void main(){
```

```
Etudiant E1 = {"Ben Salem", "Ali", {10.0, 12.5, 13.6,
7.5}, 2, 10.9};
Etudiant E2 = {"Selmi ", "Ahmed", {12.0, 12.5, 14.0,
11.5}, 2, 12.5};
strcpy (E1.nom, "Ben Arfa");
E1.notes[2] = 18;
   printf("nom : %s\n",E1.nom);
   printf("note :%f  \n", E1.notes[2]);
   for(int i=0;i<4;i++)
   printf  ("%f\t   ", E2.notes[i]);}
```

```
nom : Ben Arfa
note :18.000000
12.000000          12.500000       14.000000       11.500000
Process returned 13 (0xD)   execution time : 0.023 s
Press any key to continue.
```

Imbrication des structures(4):

Structure comportant d'autres structures:

❖ Une structure peut contenir aussi des champs de type structure.

Exemple:

```
struct Date {
 int jour;
 int mois;
 int annee; };
Typedef struct Date Date;
struct UnEtudiant {
char nom [30] ;
char prenom [30] ;
Date date_naissance ; };
Typedef  struct UnEtudiant  Etudiant;
```

Imbrication des structures(5):

Structure comportant d'autres structures:

Initialisation:

```
Etudiant E1 = {"Ali", "Ben Salem", {1, 1, 2000}};
```

Utilisation:

```
E1.date.jour = 1;
scanf ("%d %d %d", &E1.date.jour, &E1.date.mois, &E1.date.annee);
```

Exemple:

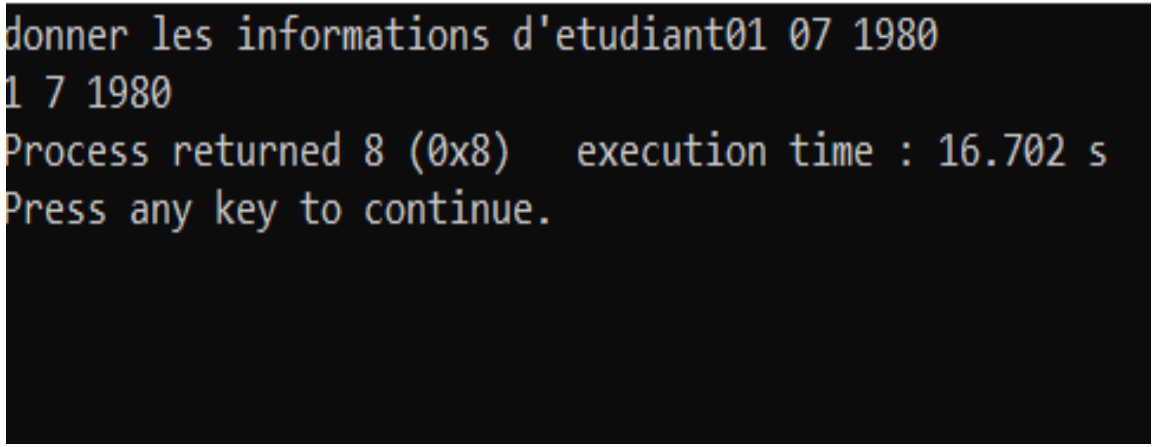
```
#include<stdio.h>
struct Date {
 int jour;
 int mois;
 int annee;
};
typedef struct Date Date;
```

Imbrication des structures(6):

Structure comportant d'autres structures:

Exemple:

```
struct UnEtudiant {
char nom [30] ;
char prenom [30] ;
Date date_naissance ; };
typedef  struct UnEtudiant  Etudiant;
void main(){
Etudiant E1 = {"Ali", "Ben Salem", {1, 1, 2000}};
printf("donner les informations d'etudiant");
scanf  ("%d \t %d \t %d", &E1.date_naissance.jour, &E1.date_naissance.mois,
&E1.date_naissance.annee);
printf("%d %d %d", E1.date_naissance.jour, E1.date_naissance.mois, E1.date_naissance.annee);
}
```



Imbrication des structures(7):

Tableaux de structures:

❖ Étant donné qu'une structure est composée d'éléments de taille fixes, il est possible de créer un tableau ne contenant que des éléments du type d'une structure donnée.

Déclaration:

Struct <nom_structure> {
<type_champ1> <nom_champ1>;
…
<type_champN> <nom_champN>;
};
Struct <nom_structure> <nom-tab> [N];

Accès:

- A un élément du tableau :

<nom-tab>[i] ;

- A un élément d'une structure :

<nom-tab>[[i]. <nom_champj>;

Imbrication des structures(8):

Tableaux de structures:

❖ Étant donné qu'une structure est composée d'éléments de taille fixes, il est possible de créer un tableau ne contenant que des éléments du type d'une structure donnée.

Déclaration:

Struct <nom_structure> {
<type_champ1> <nom_champ1>;
…
<type_champN> <nom_champN>;
};
Struct <nom_structure> <nom-tab> [N];

Accès:

- A un élément du tableau :

<nom-tab>[i] ;

- A un élément d'une structure :

<nom-tab>[[i]. <nom_champj>;

Imbrication des structures(9):

Tableaux de structures: Exemples:

```
struct Etudiant {
 char nom [30] ;
 char prenom [30] ;
 float notes [4] ;} ;
typedef  struct Etudiant  Etud;
void main(){
Etud tab [5];
for (int i = 0; i < 5; i++){
printf  ("Entrez le nom de l 'étudiant  ");
scanf  ("%s", tab[i].nom);
printf  ("Entrez le prénom de l'étudiant");
scanf  ("%s", tab[i].prenom);
 for (int j =0; j < 4; j++){
printf  ("Entrez la note %d de l'étudiant n°%d ", j+1, i+1);
 scanf  ("%f", &tab[i].notes[j]);}}}
```

Pointeurs sur structures(1):

❖ Il est possible de déclarer un pointeur sur une variable de type structure et de l'utiliser comme tout autre pointeur:

Exemple:

```
#include <stdio.h>
#include <stdlib.h>
struct DateDef  {
int day;
int month;
int year;
 };
typedef  struct DateDef  Date;
int main(){
Date *D;
Date d_Taravail;
D = &d_Taravail;
(*D).day = 21;
(*D).month = 07;
(*D).year = 1968;
printf("la date est %d %d %d
\n",(*D).day,(*D).month,(*D).year);
 ++(*D).day;
printf(" %d ",(*D).day);}
```

```
la date est 21 7 1968
 22
Process returned 0 (0x0)   execution time : 0.011 s
Press any key to continue.
```

Pointeurs sur structures(2):

- Le langage C a prévu un nouvel opérateur, noté ->:
- **(*D).day** sera exprimé plus clairement par: **D->day.**

Allocation dynamique:

```
#include <stdio.h>
#include <stdlib.h>
struct DateDef {
int day;
int month;
int year;
 };
typedef struct DateDef Date;
 int main()
 {
Date *D;
D = (struct Date*)malloc(sizeof( Date));
 D->month = 9;
 D->day = 25;
 D->year = 1983;
 printf("Todays date is %d/%d/%d.\n",
D->month, D->day, D->year % 100);
 return 0;
 }
```

```
Todays date is 9/25/83.

Process returned 0 (0x0)   execution time : 0.012 s
Press any key to continue.
```

Exercice 1:

Créer une structure fiche contenant les informations suivantes :

- Nom sur 20 caractères
- Prénom sur 20 caractères
- Age de type entier
- Note de type réel

Saisir une fiche et puis l'afficher.

Exercice 2:

Une ville a un nom, une population (nombre d'habitants) et appartient à un pays.

Ecrire un programme permettant de:

1) Déclarer la structure ville.
2) Déclarer un tableaux de villes.
3) Demander à l'utilisateur de remplir le tableau de n villes (n représente le nombre de villes d'un pays qui doit être supérieur à 0 et ne peut pas dépasser la taille maximale du tableau);
4) Afficher le tableau
5) Afficher la population totale d'un pays dont le nom est donné par l'utilisateur.
6) Afficher la ville la plus peuplée.

Exercice 3:

1) Définissez la structure « stagiaire » constituée des champs suivants:

Champ	Type
Nom	Chaine de caractère
Prénom	Chaine de caractère
Datenais	Structure

2) Le champ « Datenais » est auusi est une structure dont les champs sont:

Champ	Type
Jour	int
Mois	int
Année	int

Ecrire ensuite un programme qui permet de saisir et après afficher le nom, prénom et date de naissance d'un stagiaire.

Exercice 3:

3) On souhaite gérer les notes d'un etudiant.

Pour cela on va définir la structure « Etudiant » dont les champs sont:

Champ	Type
Nom	Chaine de caractère
Prénom	Chaine de caractère
Note	Tableau de 3 éléments
Moyenne	Float

Ecrire un programme qui permet de saisir les informations d'un étudiant (nom, prénom et notes), de calculer sa moyenne et d'afficher à la& fin un message sous la forme suivante:

« la moyenne de l'étudiant Ahmed Youssef est :12.60 »

4) Modifier le programme de la question précédent (question 3) de façon que l'on puisse gérer les notes de 50 étudiants.